

Programação e Desenvolvimento de Software I

Alocação dinâmica de memória

Prof. Héctor Azpúrua
(slides adaptados do Prof. Pedro Olmo)

Alocação Estática x Dinâmica

Alocação em C

- C tem dois tipos de alocação de memória:
 - **Estática e Dinâmica**
- Na alocação **estática**, o espaço para as variáveis é reservado no início da execução, não podendo ser alterado depois
 - `int a;`
 - `int b[20];`
- **Pergunta:** Qual o espaço de memória que deve ser reservado para um editor de texto? 🤔
 - Na alocação **dinâmica**, o espaço para as variáveis pode ser alocado durante a execução do programa

Alocação estática

- Na alocação estática, a memória é alocada **automaticamente** quando definimos uma variável
 - Neste caso, o `float` precisa de quantos bytes de memória?
 - 4!**

```
int main(int argc, char *argv[]) {  
    float x;  
    x = 3.14f;  
    return 0;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação estática

- Na alocação estática, a memória é alocada **automaticamente** quando definimos uma variável
 - Neste caso, o `float` precisa de quantos bytes de memória?
 - 4!**

```
int main(int argc, char *argv[]) {  
    float x;  
    x = 3.14f;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
x	float	1	0xAB7	
		1	0xAB8	
		1	0xAB9	
		1	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação estática

- Na alocação estática, a memória é alocada **automaticamente** quando definimos uma variável
 - Neste caso, o `float` precisa de quantos bytes de memória?
 - 4!**

```
int main(int argc, char *argv[]) {  
    float x;  
    x = 3.14f;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
x	float	1	0xAB7	3.14
		1	0xAB8	3.14
		1	0xAB9	3.14
		1	0xABA	3.14
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação Estática x Dinâmica

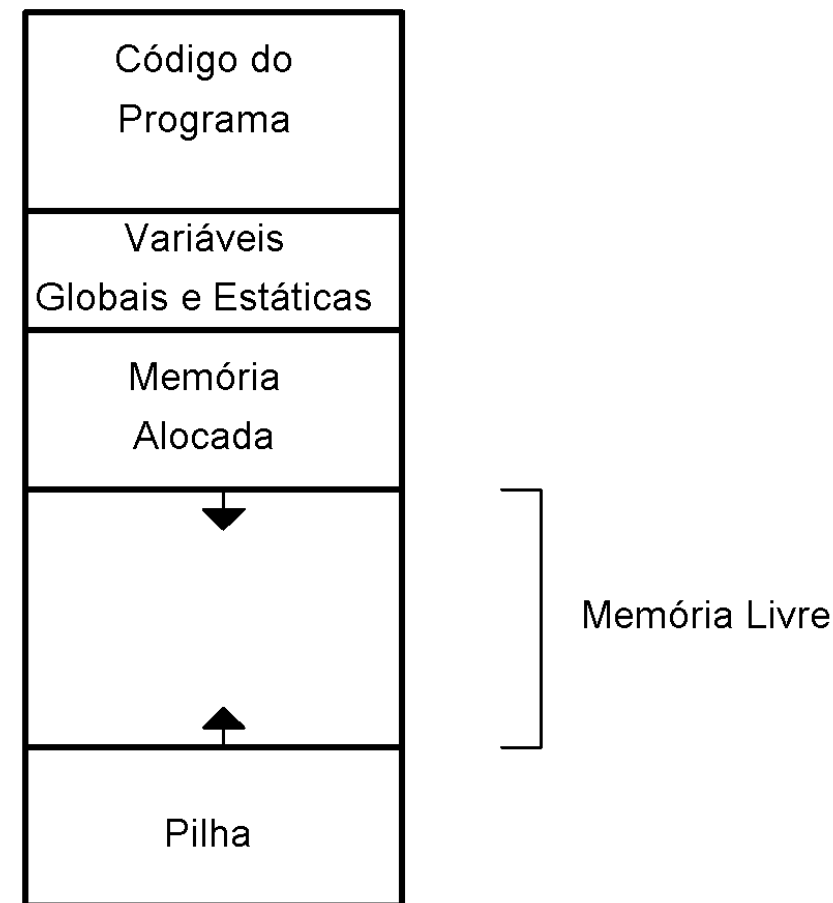
Alocação Dinâmica

- As variáveis alocadas dinamicamente são chamadas de **apontadores** (*pointers*)
 - Elas armazenam o endereço de memória de uma variável
- A memória alocada dinamicamente faz parte de uma área de memória chamada **heap**
 - Basicamente, o programa aloca e desaloca porções de memória do **heap** durante a execução

Alocação Estática x Dinâmica

Memória do sistema

- pilha/stack e o heap são segmentos lógicos de memória
 - Stack é LIFO (last in first out)
 - Tem um **limite de tamanho**
 - Libera a memória ao sair do escopo da função
- Às vezes, você não sabe quanta memória precisará:
 - Você **não quer alocar** um buffer enorme só por precaução...
- Alocar no heap permite que a quantidade de espaço de armazenamento que você tem seja alterada em tempo de execução



Esquema da memória do sistema

Alocação Estática x Dinâmica

Memória do sistema

	Stack	Heap
Velocidade	Acesso muito rápido	Acesso (relativamente) mais lento
Alocação automática	Não é necessário desalocar variáveis explicitamente	Você é responsável por alocar e liberar variáveis
Redimensionar	As variáveis não podem ser redimensionadas	Variáveis podem ser redimensionadas usando <code>realloc()</code>
Limites	Limite no tamanho da pilha (dependente do sistema operacional)	Sem limite de tamanho de memória (o limite é o tamanho total livre)
Escopo	Somente variáveis locais	Variáveis podem ser acessadas globalmente
Eficiência	O espaço é gerenciado eficientemente pela CPU, a memória não fica fragmentada	Sem garantia de uso eficiente do espaço, a memória pode ficar fragmentada ao longo do tempo

Alocação Estática x Dinâmica

Alocação Dinâmica

- `a` é um `int`
- `b` é um **apontador** para um `int`

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	11
0xAB7	b	0xFFA3
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Valor
0xFFA1	
0xFFA2	
0xFFA3	15
0xFFA4	
0xFFA5	



Alocação dinâmica

Utilizando `malloc`

- Ao utilizar ponteiros a memória **não é automaticamente alocada!**

```
int main(int argc, char *argv[]) {  
    char *nome;  
    printf("%s\n", nome);  
    return 0;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação dinâmica

Utilizando `malloc`

- Ao utilizar ponteiros a memória **não é automaticamente** alocada!
 - Como solucionar o problema?
 - Alocando a memória com `malloc`

```
int main(int argc, char *argv[]) {  
    char *nome;  
    printf("%s\n", nome);  
    return 0;  
}
```

Estamos acessando memória não alocada!

- Melhor caso: acessamos lixo inválido
- Pior caso: segmentation fault

Memória

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
nome	char *	1	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação dinâmica

Utilizando `malloc`

```
#include <stdlib.h>
```

```
void* malloc(size_t size);
```

- `size_t size`:
 - O tamanho da memória que queremos alocar em `bytes`
- `void*`
 - Retorna um ponteiro `void` para o espaço alocado ou `NULL` se não houver memória suficiente disponível
- **Importante:**
 - Não é necessário fazer **cast** no `malloc`, pois o compilador automaticamente faz a conversão por nós
 - Mas é **geralmente** aceite como uma boa prática...

```
int *idade = malloc(4);
```

```
int *idade = (int *) malloc(4);
```

← O que seriam esses 4 aí?

Cast/conversão explícita a ponteiro de inteiro

Alocação dinâmica

Utilizando `malloc` e `sizeof`

- Sempre vou precisar de lembrar do tamanho da variável para usar no `malloc`?
 - Não! Existe um operador que vai nos ajudar...

`sizeof(expression);`

- `sizeof` é um operador unário que retorna o valor em bytes do valor resultante de uma expressão ou de um tipo de dados
 - “Unary operators in C are operators that act on a single operand to perform a specific operation”

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Size of char: %lu bytes\n", sizeof(char));
```

```
    printf("Size of int: %lu bytes\n", sizeof(int));
```

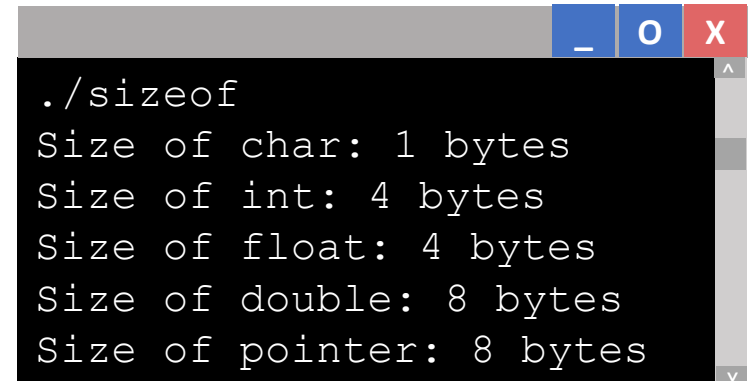
```
    printf("Size of float: %lu bytes\n", sizeof(float));
```

```
    printf("Size of double: %lu bytes\n", sizeof(double));
```

```
    printf("Size of pointer: %lu bytes\n", sizeof(void *));
```

```
    return 0;
```

```
}
```



```
./sizeof
Size of char: 1 bytes
Size of int: 4 bytes
Size of float: 4 bytes
Size of double: 8 bytes
Size of pointer: 8 bytes
```

Alocação dinâmica

Utilizando `malloc`

- Quais opções estão corretas para criar um vetor de 10 posições?

```
int N = 10;
```

- ~~1) `int *idades = (int *)malloc(N);`~~
- 2) `int *idades = (int *)malloc(sizeof(int) * N);`
- 3) `int *idades = malloc(sizeof(int) * N);`
- ~~4) `int *idades = malloc(N);`~~

- Opções 1 e 4 não estão utilizando o tamanho da variável para saber quantos bytes realmente precisamos alocar
- Lembrar de usar o `sizeof` sempre que necessário!

Apontadores

Revisão de acesso a apontadores

```
int a = 10;
```

```
int *p = &a;
```

- Acessar o valor da variável:

- Endereço de memória armazenado

```
printf("%p", p);
```

- Acessar o conteúdo:

- Valor associado ao endereço de memória armazenado

```
printf("%d", *p);
```

Alocação dinâmica

Utilizando `malloc`

- Ao utilizar ponteiros a memória **não é automaticamente alocada!**
 - Alocamos usando `malloc`

```
int main(int argc, char *argv[]) {  
    char *nome;  
    nome = malloc(sizeof(char) * 5);  
    printf("%s\n", nome);  
    return 0;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação dinâmica

Utilizando `malloc`

- Ao utilizar ponteiros a memória **não é automaticamente alocada!**
 - Alocamos usando `malloc`

```
int main(int argc, char *argv[]) {  
    char *nome;  
    nome = malloc(sizeof(char) * 5);  
    printf("%s\n", nome);  
    return 0;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
nome	char *	1	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação dinâmica

Utilizando `malloc`

- Ao utilizar ponteiros a memória **não é automaticamente alocada!**
 - Alocamos usando `malloc`

```
int main(int argc, char *argv[]) {  
    char *nome;  
    nome = malloc(sizeof(char) * 5);  
    printf("%s\n", nome);  
    return 0;  
}
```

Alocamos 5 vezes na memória o tamanho de um `char`

Memória

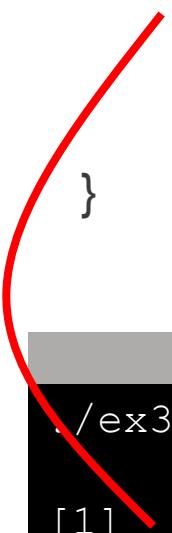
Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
nome	char *	1	0xAB7	0xAB9
		0	0xAB8	
		1	0xAB9	
		1	0xABA	
		1	0xABB	
		1	0xABC	
		1	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Alocação dinâmica

Utilizando `malloc`

- Acessando um ponteiro nulo!

```
int main(int argc, char *argv[]) {  
    float *p = NULL;  
    printf("%f", *p);  
    return 0;  
}
```



```
./ex3  
[1] 43627 segmentation fault ./ex3
```

```
int main(int argc, char *argv[]) {  
    float *p = NULL;  
    p = (float *)malloc(sizeof(float));  
    printf("%f\n", *p);  
    return 0;  
}
```

```
./ex3  
0.000000
```

Alocação dinâmica

Vantagens

- Por que precisamos de memória dinâmica?
 - Permite **alocação e liberação de memória** durante a execução de um programa
 - Alocar memória exatamente quando necessário, em vez de pré-alocá-la com uma declaração de matriz de tamanho específico (gigante)
 - Uso da memória/variáveis **fora do escopo da função!**
- Quando usar o *heap*?
 - Alocar um grande pedaço de memória (ex. array ou struct de grande tamanho)
 - Manter a variável viva por um longo período de tempo (ex. variáveis globais)
 - Vetores e matrizes que precisam trocar de tamanho
 - Uso de variáveis fora do escopo da função original

Alocação dinâmica

Definição de variável dentro de uma função

```
struct Pessoa {  
    char *nome;  
    int idade;  
};  
  
struct Pessoa *criar_pessoa1(const char *nome, int idade) {  
    struct Pessoa p;  
    strcpy(p.nome, nome);  
    p.idade = idade;  
    return &p;  
}  
  
int main(int argc, char *argv[]) {  
    struct Pessoa *p = criar_pessoa1("Joao", 20);  
    printf("%s, %d\n", p->nome, p->idade);  
    return 0;  
}
```

```
./e3  
[1] 43627 segmentation fault ./e3
```



Alocação dinâmica

Definição de variável dentro de uma função

- O que aconteceu?

- A função `criar_pessoa` criou a variável na *pilha/Stack*
- Isso significa que ao sair da função essa memória é liberada!

- Mas eu retornei um ponteiro!

- Sim, mas ele **já não aponta a mais nada**
- Por isso o erro de *segmentação* ao tentar acessar o valor do ponteiro...

- Como solucionar?

```
struct Pessoa {  
    char *nome;  
    int idade;  
};  
  
struct Pessoa *criar_pessoa1(  
    const char *nome, int idade) {  
  
    struct Pessoa p;  
    strcpy(p.nome, nome);  
    p.idade = idade;  
    return &p;  
}
```



Alocação dinâmica

Definição de variável dentro de uma função

```
struct Pessoa {  
    char *nome;  
    int idade;  
};  
  
struct Pessoa *criar_pessoa2(const char *nome, int idade) {  
    struct Pessoa *p = (struct Pessoa *)malloc(sizeof(struct Pessoa));  
    p->nome = malloc(sizeof(char) * 50);  
    strcpy(p->nome, nome);  
    p->idade = idade;  
    return p;  
}  
  
int main(int argc, char *argv[]) {  
    struct Pessoa *p = criar_pessoa2("Joao", 20);  
    printf("%s, %d\n", p->nome, p->idade);  
    return 0;  
}
```

A terminal window with a dark background and a title bar containing a minus sign, a maximize button, and a close button. The terminal shows the command './e3' on the first line and the output 'Joao, 20' on the second line.

```
./e3  
Joao, 20
```



Liberação de Memória

Importância

- O código anterior tem um problema...
- A memória deve ser liberada após o término de seu uso
 - **Por que?** 🤔
- A liberação deve ser feita por quem fez a alocação:
 - **Estática:** compilador
 - **Dinâmica:** programador (nós!)
- Como liberamos a memória?
 - Operador `free(ponteiro);`

Alocação dinâmica

Definição de variável dentro de uma função

```
struct Pessoa {  
    char *nome;  
    int idade;  
};  
struct Pessoa *criar_pessoa2(const char *nome, int idade) {  
    struct Pessoa *p = (struct Pessoa *)malloc(sizeof(struct Pessoa));  
    p->nome = malloc(sizeof(char) * 50);  
    strcpy(p->nome, nome);  
    p->idade = idade;  
    return p;  
}  
int main(int argc, char *argv[]) {  
    struct Pessoa *p = criar_pessoa2("Joao", 20);  
    printf("%s, %d\n", p->nome, p->idade);  
    if (p != NULL) {  
        free(p->nome);  
        free(p);  
    }  
    return 0;  
}
```

Primeiro desalocamos
nome e **depois** a
estrutura p



Apontadores e memória dinâmica

Notação

- Definição de `p` como um apontador para uma variável do tipo `tipo`:
 - `tipo *p;`
- Alocação de memória para uma variável apontada por `p`:
 - `p = (tipo*) malloc(sizeof(tipo));`
- Liberação de memória:
 - `free(p);`
- Conteúdo da variável apontada por `p`:
 - `*p;`
- Valor nulo para um apontador:
 - `NULL;`
- Endereço de uma variável `a`:
 - `&a;`

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30; // qual o valor de b?  
  
    return 0;  
}
```

Memória estática / Stack

Endereço	Variável	Valor
0xAB6		
0xAB7		
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	0	
0xFFA2	0	
0xFFA3	0	
0xFFA4	0	
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30; // qual o valor de b?  
  
    return 0;  
}
```

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xFFA1
0xAB7	b	10
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	
0xFFA2	1	
0xFFA3	1	
0xFFA4	1	
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30; // qual o valor de b?  
  
    return 0;  
}
```

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xFFA1
0xAB7	b	10
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30; // qual o valor de b?  
  
    return 0;  
}
```

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xAB7
0xAB7	b	10
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30; // qual o valor de b?  
  
    return 0;  
}
```

A memória que foi alocada no *heap*
Ainda continua alocada!

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xAB7
0xAB7	b	30
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Exemplo 2

- O que aconteceu?
 - A memória alocada no *heap* precisa ser explicitamente liberada!
 - Mesmo saindo da função ou redefinindo o ponteiro que apontava ao endereço de memória dessa variável...
- Se esquecermos libera-la então teremos o que é chamado de **leak de memória!**
- **Como solucionar?**

Heap		
Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30;  
  
    free(b);  
  
    return 0;  
}
```



Não posso fazer isso
pois b não é ponteiro!

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xAB7
0xAB7	b	30
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;  
    a = &b;  
    *a = 30;  
  
    free(a);  
  
    return 0;  
}
```

a já não aponta mais para
a memória no *heap*...
Esta apontando para a
memória estática!

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xAB7
0xAB7	b	30
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Exemplo 2

```
int main(int argc, char *argv[]) {  
    int *a, b;  
    b = 10;  
    a = (int *)malloc(sizeof(int));  
    *a = 20;
```

```
    free(a);
```

 Aqui sim!

```
    a = &b;  
    *a = 30;  
    return 0;
```

```
}
```

Memória estática / Stack

Endereço	Variável	Valor
0xAB6	a	0xAB7
0xAB7	b	30
0xAB8		
0xAB9		
0xABA		

Heap

Endereço	Aloc?	Valor
0xFFA1	1	20
0xFFA2	1	20
0xFFA3	1	20
0xFFA4	1	20
0xFFA5	0	

Alocação dinâmica

Erros comuns

- Esquecer de alocar memória e tentar acessar o conteúdo da variável
- Copiar o valor do apontador ao invés do valor da variável apontada
- Esquecer de desalocar memória
 - Ela é desalocada ao fim do programa, mas pode ser um problema em loops
- **Tentar acessar o conteúdo da variável depois de desalocá-la**

Alocação dinâmica de vetores

- Normalmente, a alocação dinâmica é utilizada para criar vetores em tempo de execução

- Exemplo:

```
int *p;
```

```
p = (int *)malloc(10*sizeof(int));
```

- Aloca um vetor de inteiros com 10 posições. A manipulação pode ser feita normalmente: $p[i] = \dots$
- O apontador **p** guarda o endereço (aponta) da primeira posição do vetor

Alocação dinâmica de vetores

Vetores e apontadores

- Em C todo vetor (mesmo alocados de forma estática) é um **apontador para sua primeira posição**
- Pode se trabalhar usando ambas notações:
 - $*p$ é equivalente a $p[0]$
 - p é equivalente a $\&p[0]$
 - $*(p + i)$ é equivalente a $p[i]$
 - Considerando v um vetor alocado estaticamente, e p dinamicamente:
 - Pode-se fazer $p = v$
 - Mas não $v = p$ (v é, de certa forma, um “ponteiro constante”)

Alocação dinâmica de vetores

Vetores e apontadores

- Qual a saída dos programas abaixo?

```
int main(int argc, char *argv[]) {  
    int a[10], *b;  
    b = a;  
    b[5] = 999;  
    printf("%d\n", a[5]);  
    printf("%d\n", b[5]);  
  
    return 0;  
}
```



```
./ex3  
999  
999
```

```
int main(int argc, char *argv[]) {  
    int a[10], *b;  
    b = (int *)malloc(sizeof(int) * 10);  
    b[5] = 999;  
    printf("%d\n", a[5]);  
    printf("%d\n", b[5]);  
    free(b);  
    return 0;  
}
```



```
./ex3  
268435455  
999
```

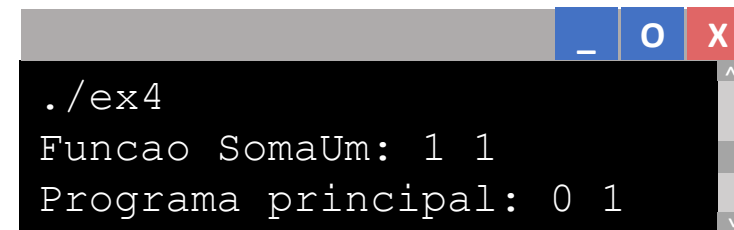
Passagem de parâmetros

- Em C só existe passagem por valor, logo deve-se implementar a passagem por referência utilizando-se apontadores

```
#include <stdio.h>
```

```
void SomaUm(int x, int *y) {  
    x = x + 1;  
    *y = (*y) + 1;  
    printf("Funcao SomaUm: %d %d\n", x, *y);  
}
```

```
int main() {  
    int a = 0, b = 0;  
    SomaUm(a, &b);  
    printf("Programa principal: %d %d\n", a, b);  
}
```

A terminal window with a black background and white text. The title bar shows a minus sign, a maximize button, and a close button. The terminal content shows the command './ex4' being executed, followed by two lines of output: 'Funcao SomaUm: 1 1' and 'Programa principal: 0 1'.

```
./ex4  
Funcao SomaUm: 1 1  
Programa principal: 0 1
```

Passagem de parâmetros

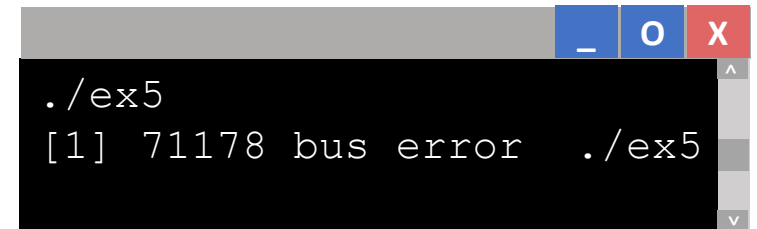
Alocando memória dentro do procedimento

- E para alocar memória dentro de um procedimento?
 - Esse código aqui vai funcionar?

```
#include <stdlib.h>
```

```
void aloca(int *x, int n) {  
    x = (int *)malloc(n * sizeof(int));  
    x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(a, 10);  
    a[1] = 40;  
}
```



A terminal window with a black background and white text. The window has a title bar with three buttons: a minus sign, a circle, and an 'X'. The text inside the terminal shows the command `./ex5` being executed, followed by an error message: `[1] 71178 bus error ./ex5`. The terminal has scrollbars on the right side.

Passagem de parâmetros

Exemplo com ponteiros simples

- que acontece com a memória?

```
#include <stdlib.h>
```

```
void aloca(int *x, int n) {  
    x = (int *)malloc(n * sizeof(int));  
    x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(a, 5);  
    a[1] = 40;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo com ponteiros simples

- que acontece com a memória?

```
#include <stdlib.h>
```

```
void aloca(int *x, int n) {  
    x = (int *)malloc(n * sizeof(int));  
    x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(a, 5);  
    a[1] = 40;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	1	0xAB6	NULL
x	int *	1	0xAB7	NULL
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo com ponteiros simples

- que acontece com a memória?

```
#include <stdlib.h>
```

```
void aloca(int *x, int n) {  
    x = (int *)malloc(n * sizeof(int));  
    x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(a, 5);  
    a[1] = 40;  
}
```

Memória

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	1	0xAB6	NULL
x	int *	1	0xAB7	0xAB8
		1	0xAB8	20
		1	0xAB9	
		1	0xABA	
		1	0xABB	
		1	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo com ponteiros simples

- que acontece com a memória?

```
#include <stdlib.h>
```

```
void aloca(int *x, int n) {  
    x = (int *)malloc(n * sizeof(int));  
    x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(a, 5);  
    a[1] = 40;  
}
```



Memória

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	1	0xAB6	NULL
		0	0xAB7	
		1	0xAB8	20
		1	0xAB9	
		1	0xABA	
		1	0xABB	
		1	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Alocando memória dentro do procedimento

- Para alocar memória dentro de um procedimento:
 - Basta passar a variável (apontador) como referência
 - No entanto, como não há passagem por referência as coisas são um pouco mais complicadas

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {  
    *x = (int *)malloc(n * sizeof(int));  
    *x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(&a, 10);  
    a[1] = 40;  
}
```



Passagem de parâmetros

Exemplo

- Nesse caso precisamos usar um **ponteiro de ponteiro** (`int **`)

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {  
    *x = (int *)malloc(n * sizeof(int));  
    *x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(&a, 5);  
    a[1] = 40;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo

- Nesse caso precisamos usar um **ponteiro de ponteiro** (`int **`)

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {  
    *x = (int *)malloc(n * sizeof(int));  
    *x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(&a, 5);  
    a[1] = 40;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	0	0xAB6	NULL
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo

- Nesse caso precisamos usar um **ponteiro de ponteiro** (`int **`)

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {  
    *x = (int *)malloc(n * sizeof(int));  
    *x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(&a, 5);  
    a[1] = 40;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	0	0xAB6	NULL
x	int **	0	0xAB7	0xAB6
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo

- Nesse caso precisamos usar um **ponteiro de ponteiro** (`int **`)

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {  
    *x = (int *)malloc(n * sizeof(int));  
    *x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(&a, 5);  
    a[1] = 40;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	0	0xAB6	0xAB8
x	int **	0	0xAB7	0xAB6
		1	0xAB8	
		1	0xAB9	
		1	0xABA	
		1	0xABB	
		1	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo

- Nesse caso precisamos usar um **ponteiro de ponteiro** (`int **`)

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {  
    *x = (int *)malloc(n * sizeof(int));  
    *x[0] = 20;  
}
```

```
int main() {  
    int *a = NULL;  
    aloca(&a, 5);  
    a[1] = 40;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
a	int *	0	0xAB6	0xAB8
x	int **	0	0xAB7	0xAB6
		1	0xAB8	20
		1	0xAB9	
		1	0xABA	
		1	0xABB	
		1	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Passagem de parâmetros

Exemplo

- Nesse caso precisamos usar um **ponteiro de ponteiro** (`int **`)

```
#include <stdlib.h>
```

```
void aloca(int **x, int n) {
    *x = (int *)malloc(n * sizeof(int));
    *x[0] = 20;
}
```

```
int main() {
    int *a = NULL;
    aloca(&a, 5);
    a[1] = 40;
}
```

Memoria

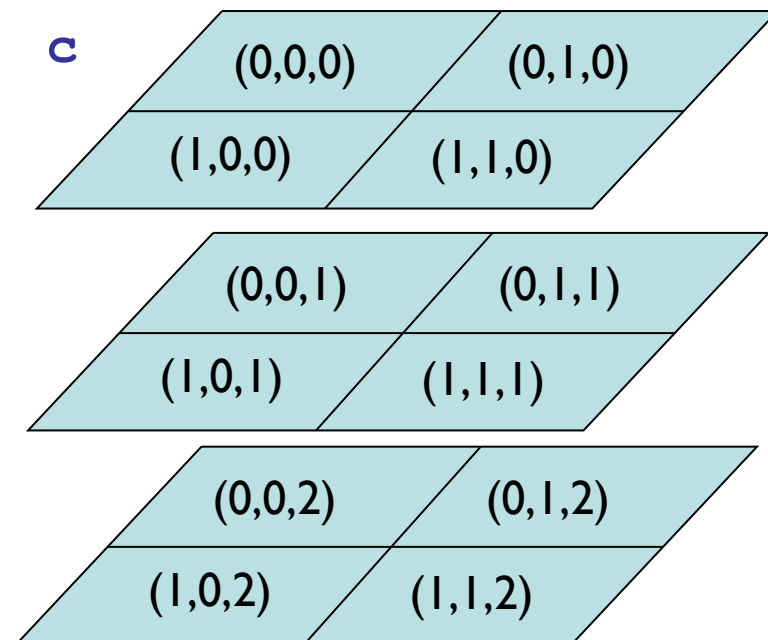
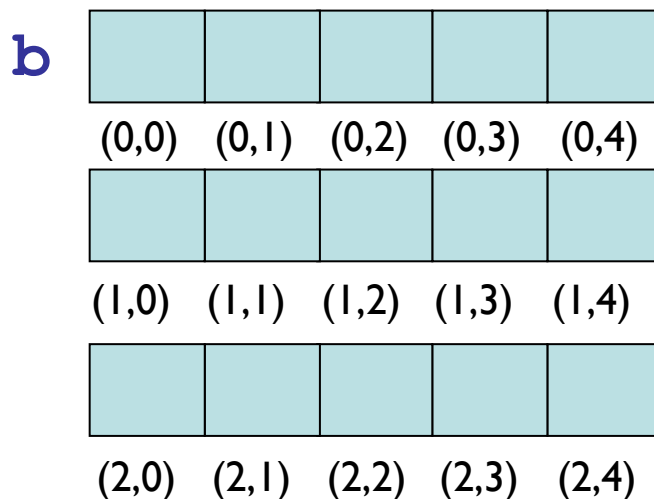
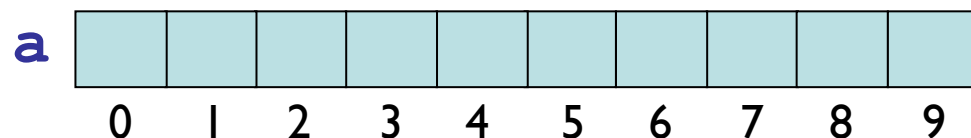
Variável	Tipo	Aloc?	Endereço	Valor
a	int *	0	0xAB6	0xAB8
		0	0xAB7	
		1	0xAB8	20
		1	0xAB9	40
		1	0xABA	
		1	0xABB	
		1	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	

Vetores e matrizes

Relembrando...

- Em programação, os termos **vetor** e **matriz** são usados como sinônimos de variável indexada
- O termo **vetor** é usado para uma variável indexada de uma única dimensão e o termo **matriz** para variáveis indexadas de duas ou mais dimensões

```
int a[10];  
char b[3][5];  
double c[2][2][3];
```



Representação de vetores

Exemplo com strings

- `str` é um ponteiro ao começo da memória contigua de 5 caracteres

```
int main() {  
    char c = 'A';  
    char str[] = "OLA!";  
  
    *(str + 0) = 'a';  
    *(str + 2) = 'b';  
  
    str[1] = 'Z';  
    return 0;  
}
```

- E uma matriz?

Endereço	Variável	Conteúdo
0xFF1	c	'A'
0xFF2		
0xFF3	str[0]	'a'
0xFF4	str[1]	'Z'
0xFF5	str[2]	'b'
0xFF6	str[3]	'!'
0xFF7	str[4]	'\0'
0xFF8		
0xFF9		
0xFFA		



Alocação dinâmica de matrizes

O que é uma matriz?

- Ao executar: `int c[3][3];`
 - Automaticamente a memória referente a $(3 * 3) * 4$ bytes é alocada de forma estática
- Se para criar um vetor dinamicamente usamos:

```
int size = 50;  
char *nome;  
nome = malloc(sizeof(char) * size);
```

- Daria para usar o mesmo código para alocar uma **matriz**? 🤔

Alocação dinâmica de matrizes

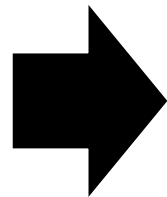
O que é uma matriz?

- Se um **vetor** é um **ponteiro**, então:
 - Uma **matriz** é na verdade um **ponteiro para ponteiros!** (Ou vetor de vetores)

```
int **matriz_1 = NULL;
```

Alocar matriz no Stack
(memória *estática*)

```
int main() {  
    int n = 3;  
    int m = 2;  
  
    int matriz_1[n][m];  
    return 0;  
}
```



Alocar matriz no Heap
(memória *dinâmica*)

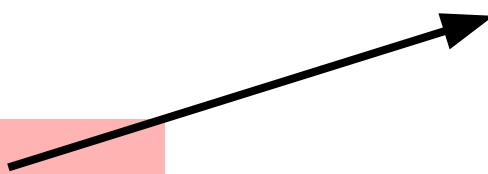
```
int main() {  
    int n = 3;  
    int m = 2;  
    int **matriz_2 = NULL;  
    matriz_2 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        matriz_2[i] = (int *) malloc(m * sizeof(int));  
    return 0;  
}
```

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Matriz 3x2



0	1
2	3
4	5

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
		0	0xAB6	
		0	0xAB7	
		0	0xAB8	
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	
		0	0xAC2	
		0	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	NULL
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
		0	0xAC1	
		0	0xAC2	
		0	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

O que seria isso aqui?



Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
		0	0xAB9	
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
m1[0]	int*	1	0xAC1	
m1[1]	int*	1	0xAC2	
m1[2]	int*	1	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
i	int	1	0xAB9	0
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
		0	0xABF	
		0	0xAC0	
m1[0]	int*	1	0xAC1	
m1[1]	int*	1	0xAC2	
m1[2]	int*	1	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
i	int	1	0xAB9	0
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
m1[0][0]	int	1	0xABF	
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	
m1[2]	int*	1	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
i	int	1	0xAB9	1
		0	0xABA	
		0	0xABB	
		0	0xABC	
		0	0xABD	
m1[0][0]	int	1	0xABF	
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	
m1[2]	int*	1	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
i	int	1	0xAB9	1
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	
		0	0xABC	
		0	0xABD	
m1[0][0]	int	1	0xABF	
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
i	int	1	0xAB9	2
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	
		0	0xABC	
		0	0xABD	
m1[0][0]	int	1	0xABF	
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
i	int	1	0xAB9	2
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	
m1[2][0]	int	1	0xABC	
m1[2][1]	int	1	0xABD	
m1[0][0]	int	1	0xABF	
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	0xABC

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Para onde aponta
esse endereço?

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
		0	0xAB9	
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	
m1[2][0]	int	1	0xABC	
m1[2][1]	int	1	0xABD	
m1[0][0]	int	1	0xABF	
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	0xABC

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
		0	0xAB9	
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	
m1[2][0]	int	1	0xABC	
m1[2][1]	int	1	0xABD	
m1[0][0]	int	1	0xABF	11
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	0xABC

Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
		0	0xAB9	
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	66
m1[2][0]	int	1	0xABC	
m1[2][1]	int	1	0xABD	
m1[0][0]	int	1	0xABF	11
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	0xABC

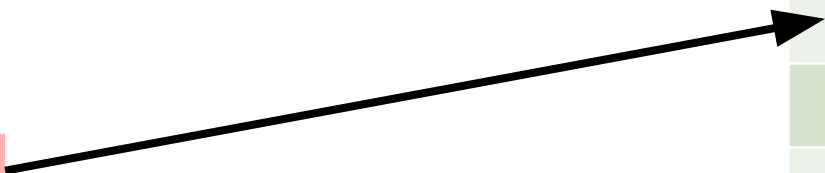
Alocação dinâmica de matrizes

O que esta acontecendo na memoria?

```
int main() {  
    int n = 3;  
    int m = 2;  
    int **m1 = NULL;  
    m1 = (int *) malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m1[i] = (int *) malloc(m * sizeof(int));  
  
    m1[0][0] = 11;  
    m1[1][1] = 66;  
    m1[2][1] = 99;  
    return 0;  
}
```

Memoria

Variável	Tipo	Aloc?	Endereço	Valor
m	int	1	0xAB6	3
n	int	1	0xAB7	2
m1	int**	1	0xAB8	0xAC1
		0	0xAB9	
m1[1][0]	int	1	0xABA	
m1[1][1]	int	1	0xABB	66
m1[2][0]	int	1	0xABC	
m1[2][1]	int	1	0xABD	99
m1[0][0]	int	1	0xABF	11
m1[0][1]	int	1	0xAC0	
m1[0]	int*	1	0xAC1	0xABF
m1[1]	int*	1	0xAC2	0xABA
m1[2]	int*	1	0xAC3	0xABC



Retorno de variáveis

Alocadas static vs dynamic

- O que vai imprimir esse programa?
 - Vai retornar o ponteiro de **segundos** nos dois casos?

```
./e1
retorna_p_v1: '1749085164'    0x16d46eacc
retorna_p_v2: '1749085164'    0x141605ea0
```

- No primeiro caso ele desalocou a memória na qual o ponteiro estava apontando..
 - Comportamento indefinido!
- No segundo caso a memória é mantida pelo uso do *heap*

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int *retorna_p_v1() {
    int segundos = time(NULL);
    printf("retorna_p_v1: '%d'\t%p\n",
           segundos, &segundos);
    return &segundos;
}

int *retorna_p_v2() {
    int *p = (int *)malloc(sizeof(int));
    *p = time(NULL);
    printf("retorna_p_v2: '%d'\t%p\n", *p, p);
    return p;
}
```

```
int main() {
    int *num1 = retorna_p_v1();
    int *num2 = retorna_p_v2();
    printf("Numero 1: '%d'\t%p\n", *num1, num1);
    printf("Numero 2: '%d'\t%p\n", *num2, num2);
    free(num2);
    return 0;
}
```

Retorno de variáveis

Vetores alocados dinamicamente

- O que vai imprimir esse programa?
 - Vai retornar o **vetor preenchido** nos dois casos?

```
./e2
retorna_v1: [1,2,3]      0x16d82eaa8
retorna_v2: [1,2,3]      0x129e05ea0
```

- Igual que no caso anterior, a memória alocada no *Stack* é desalocada ao sair do escopo da função
- Por que estamos retornando um **ponteiro de int** no caso do **vetor**?

```
int *retorna_v1() {
    int num[] = {1, 2, 3};
    printf("retorna_v1: [%d,%d,%d]\t%p\n",
           num[0], num[1], num[2], num);
    return num;
}

int *retorna_v2() {
    int *p = (int *)malloc(sizeof(int) * 3);
    for (int i = 0; i < 3; i++)
        p[i] = i + 1;
    printf("retorna_v2: [%d,%d,%d]\t%p\n",
           p[0], p[1], p[2], p);
    return p;
}

int main() {
    int *v1 = retorna_v1();
    int *v2 = retorna_v2();

    printf("Vec 1: [%d,%d,%d]\t%p\n",
           v1[0], v1[1], v1[2], v1);
    printf("Vec 2: [%d,%d,%d]\t%p\n",
           v2[0], v2[1], v2[2], v2);

    free(v2);
    return 0;
}
```

Retorno de variáveis

Matrizes alocadas dinamicamente

- Considerando que uma matriz é um **vetor de vetores**:

```
char b[3][5];  
double c[2][2][3];
```

- E um **vetor** pode ser retornado como um **ponteiro**:

```
int *retorna_p_v2() {  
    int *p = (int *)malloc(sizeof(int));  
    *p = time(NULL);  
    printf("retorna_p_v2: '%d'\t%p\n", *p, p);  
    return p;  
}
```

- Como podemos inicializar uma **matriz** e **retornar ela**?
 - Retornando um ponteiro de ponteiros!

Retorno de variáveis

Matrizes alocadas dinamicamente

- O que vai imprimir esse programa?
 - Ele vai funcionar e retornar a matriz preenchida corretamente?

```
./e3
Vec 1: [1,2,3] 0x1497040a0
Vec 2: [4,5,6] 0x1497040b0
Vec 3: [7,8,9] 0x1497040c0
```

```
int **retorna_mat() {
    int **mat = (int **)malloc(sizeof(int *) * 3);
    int count = 0;
    for (int i = 0; i < 3; i++) {
        mat[i] = (int *)malloc(sizeof(int) * 3);
        for (int j = 0; j < 3; j++) {
            mat[i][j] = ++count;
        }
    }
    return mat;
}

int main() {
    int **mat = retorna_mat();

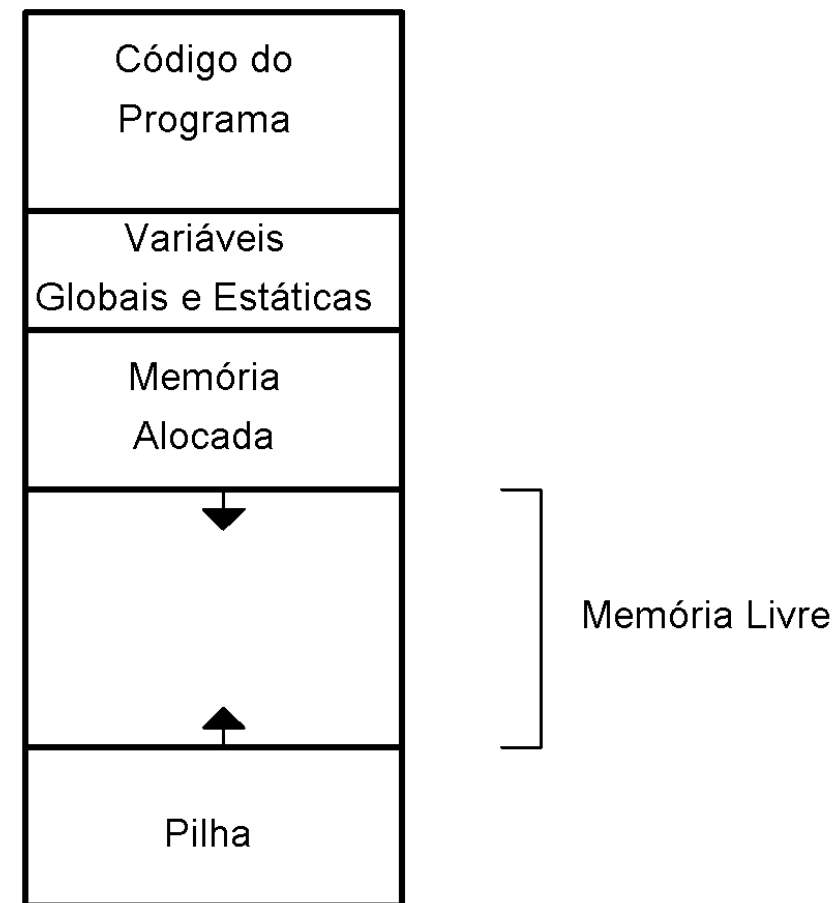
    for (int i = 0; i < 3; i++) {
        printf("Vec %d: [%d,%d,%d]\t%p\n",
            i + 1,
            mat[i][0], mat[i][1], mat[i][2], mat[i]);
    }

    for (int i = 0; i < 3; i++) {
        free(mat[i]);
    }
    free(mat);
    return 0;
}
```

Alocação Estática x Dinâmica

Memória do sistema

- pilha/stack e o heap são segmentos lógicos de memória
 - Stack é LIFO (last in first out)
 - Tem um **limite de tamanho**
 - Libera a memória ao sair do escopo da função
- Às vezes, você não sabe quanta memória precisará:
 - Você **não quer alocar** um buffer enorme só por precaução...
- Alocar no heap permite que a quantidade de espaço de armazenamento que você tem seja alterada em tempo de execução
- **Como podemos testar esses limites de memória?** 😊



Esquema da memória do sistema

Trivia: tamanho do *heap*

Será que ele realmente é limitado?

- Vamos ver o que acontece...

```
int main() {  
    int n = 1000;  
    int m_stack[n][n];  
    memset(m_stack, 0, n * sizeof(int));  
  
    printf("Size of m_stack: %zuMB\n",  
          sizeof(m_stack) / (1024 * 1024));  
    system("read -p 'Press Enter to continue...'");  
    return 0;  
}
```



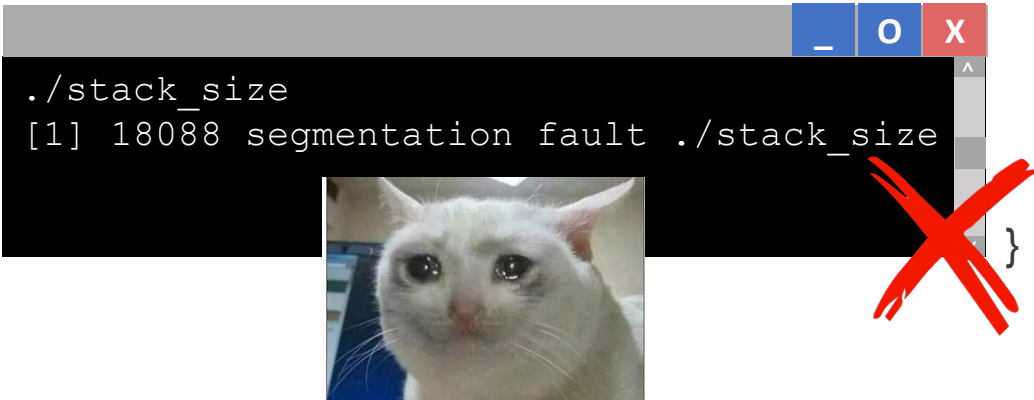
```
./stack_size  
Size of m_stack: 3MB  
Press Enter to continue...
```

Trivia: tamanho do *heap*

Será que ele realmente é limitado?

- Vamos ver o que acontece...

```
int main() {  
    int n = 2000;  
    int m_stack[n][n];  
    memset(m_stack, 0, n * sizeof(int));  
  
    printf("Size of m_stack: %zuMB\n",  
          sizeof(m_stack) / (1024 * 1024));  
    system("read -p 'Press Enter to continue...'");  
    return 0;  
}
```



Trivia: tamanho do *heap*

Será que ele realmente é limitado?

- Vamos ver o que acontece...

```
./stack_size
[1] 18088 segmentation fault ./stack_size
```

```
int main() {
    int n = 2000;
    int m_stack[n][n];
    memset(m_stack, 0, n * sizeof(int));

    printf("Size of m_stack: %zuMB\n",
           sizeof(m_stack) / (1024 * 1024));
    system("read -p 'Press Enter to continue...'");
    return 0;
}
```

```
./stack_size
Size of m_heap: 15 MB
Press Enter to continue...
```



```
int main() {
    int n = 2000;
    int **m_heap = (int **)malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++)
        m_heap[i] = (int *)malloc(n * sizeof(int));

    printf("Size of m_heap: %luMB\n",
           sizeof(int) * n * n / (1024 * 1024));
    system("read -p 'Press Enter to continue...'");
    return 0;
}
```

Trivia: tamanho do *heap*

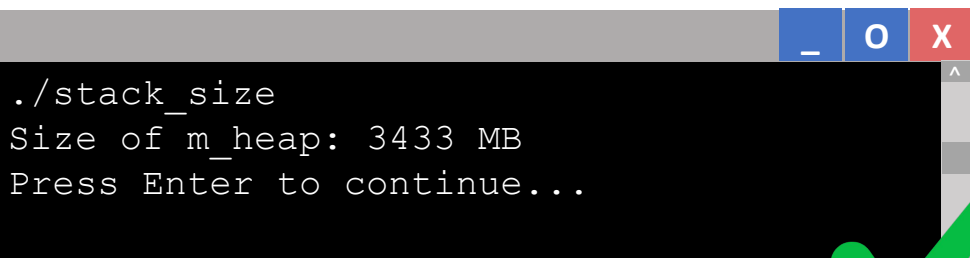
Será que ele realmente é limitado?

- Vamos ver o que acontece...

```
int main() {  
    int n = 2000;  
    int m_stack[n][n];  
    memset(m_stack, 0, n * sizeof(int));  
  
    printf("Size of m_stack: %zuMB\n",  
          sizeof(m_stack) / (1024 * 1024));  
    system("read -p 'Press Enter to continue...'");  
    return 0;  
}
```

```
int main() {  
    int n = 30000;  
    int **m_heap = (int **)malloc(n * sizeof(int *));  
    for (int i = 0; i < n; i++)  
        m_heap[i] = (int *)malloc(n * sizeof(int));  
  
    printf("Size of m_heap: %luMB\n",  
          sizeof(int) * n * n / (1024 * 1024));  
    system("read -p 'Press Enter to continue...'");  
    return 0;  
}
```

Pode isso??!



```
./stack_size  
Size of m_heap: 3433 MB  
Press Enter to continue...
```

Perguntas?

- E-mail:
 - hector@dcc.ufmg.br
- Material da disciplina:
 - <https://pedroolmo.github.io/teaching/pdsI.html>
- Github:
 - <https://github.com/h3ct0r>



Héctor Azpúrua
h3ct0r